```
model :user_preferences

# . . .
```

However, if the session might get read by another controller (which is likely in any application with multiple controllers), you'll probably want to add the declaration to `application_controller.rb` in `app/controllers`.

- You probably don't want to store massive objects in session data—put them in the database and reference them from the session. This is particularly true for cookie-based sessions, where the overall limit is 4 KB.

- You probably don't want to store volatile objects in session data. For example, you might want to keep a tally of the number of articles in a blog and store that in the session for performance reasons. But if you do that, the count won't get updated if some other user adds an article.

  It's tempting to store objects representing the currently logged-in user in session data. This might not be wise if your application needs to be able to invalidate users. Even if a user is disabled in the database, their session data will still reflect a valid status.

  Store volatile data in the database, and reference it from the session instead.

- You probably don't want to store critical information solely in session data. For example, if your application generates an order confirmation number in one request and stores it in session data so that it can be saved to the database when the next request is handled, you risk losing that number if the user deletes the cookie from their browser. Critical information needs to be in the database.

One more caveat—and it's a big one. If you store an object in session data, then the next time you come back to that browser, your application will end